Complete companion: adaptive decision support under frozen rules

Reader, purpose, and exact basis

This companion explains *Adaptive decision support under frozen rules: guarantees, failure modes, and component admission*, by Patrick Iben, as submitted to *PLOS ONE* on 11 November 2026. It restates the paper and adds no results. The concise study guide covers the same ground in four pages.

Source: the 22-page manuscript PDF with SHA-256 `5c984ac151e9751486bc5fcce03a5b8333524a2dcffd10b02b9a4fe1fc613130`. Line numbers below are that PDF's printed line numbers. The S1 to S7 Appendix files are hashed in ARTIFACTS.json.

The intended reader has taken one course in probability or statistics. No background in control theory, reinforcement learning, or software testing is assumed. Read the guided lessons (P01 to P10) first. Use the section readings (R01 to R12) and supplement modules (S01 to S07) for reference, work the six examples (W01 to W06), and then try the self-test (T01 to T12).

What was checked, and what was not. Every number in this companion and the study guide was checked by a script against the manuscript and its appendices; illustrative inputs in worked examples are labeled as such. No reader from the intended audience has reviewed this text, no person has reviewed it for claims it might add, and the self-test has not been given to readers. Those human checks are still open.

How evidence is labeled

Every lesson states its evidence class:

- `ANALYTIC`: a result proved from stated assumptions.
- `MODELED_LOCKED`: a simulation result from frozen code and pre-specified criteria.
- `MEASURED_DATED`: a measurement of the software itself, such as a test count or a mutation result.
- `DEFINITION`: what a term or component means in this study.
- `SCOPE`: what the study does or does not cover.
- `INTERPRETIVE`: the author's reading, analogy, or synthesis.
- `OPEN`: a question the study leaves untested.

A result designed after earlier results were read is marked **post hoc**. Post-hoc results are weaker evidence, and no label promotes them.

Part 1: Guided lessons (P01–P10)

P01. Two duties that conflict

Where we are: Introduction, lines 1–58.
What the passage is doing: It sets out the problem, places the study next to fault-tolerant control and runtime assurance, and states three questions.
In ordinary language: A system that keeps learning after release should improve where improvement is safe, such as the order of its suggestions. It should also hold certain rules fixed. Many reports that a dangerous shortcut worked should not teach it that shortcut. Control engineering faces a similar conflict. The Simplex architecture pairs a high-performance controller that carries no safety guarantee with a simple safe controller and a monitor that decides which one acts. This study tested a decision-support system whose anchor named rules that learning could not change. It asked what stays fixed, when learning helps and what protection costs, and when an optional component should be admitted. The primary estimand was the difference in expected net utility per episode between the full system and a simpler system held to the same bound.
Evidence status: `SCOPE` for the questions and design; `INTERPRETIVE` for the comparison with control engineering, which the paper calls an analogy.
Do not accidentally conclude: The study did not test a controller or any physical system. It tested one decision-support implementation in simulation.
Try it: Using the paper's example, name one thing the system should learn and one thing it should never learn.

P02. Eight levels, one anchor, three tiers

Where we are: Study objects and scope, and Architecture, lines 60–112, with the trace in Table 1.
What the passage is doing: It describes exactly what was tested.
In ordinary language: The system worked in eight levels, from Level 0 (Constitution and safety anchor) to Level 7 (Knowledge). Levels 0 and 7 changed only by human release. The anchor was one document that fixed how every decision was framed. Its SHA-256 hash was compiled into the code, and the system refused to start if the file differed. Three response tiers shared the anchor: Reflex could only block, redirect, give provisional guidance, or escalate; Core was the deterministic path; Max added optional adapters under the same frozen checks. The frozen checks ran before any option was ranked. The safety check blocked or redirected unsafe inputs, and the constraint check vetoed plans whose materials or hazards made them unsafe. No generative language model wrote plans or answers. Table 1 follows one problem through the levels: a warm water hose leak, with duct tape and rope available. Both physical plans used rope, rated 1 psi against the 15 psi the answer implied, so both failed and the system recommended the stop-and-monitor fallback.
Evidence status: `DEFINITION`.
Do not accidentally conclude: The levels' update frequencies were design declarations, not enforced limits. A permission function that would have enforced them was tested but was not on the live update path. A pin also protects a file without showing that its content is right.
Try it: Which two levels were frozen, and what was the only way either one could change?

P03. The update rule and its two scales (Proposition 1)

Where we are: Update rule and guarantees, lines 113–168, equations (1) and (2), and Proposition 1.
What the passage is doing: It defines how learning moved each adaptive parameter and proves that the movement stays bounded.
In ordinary language: Each outcome produced a residual, the observed value minus the predicted score. Three momentum buffers smoothed the residual at short, medium, and long scales. Their weighted sum, times a learning rate λ, gave the update. If the update opposed the slow direction stored in Low memory, it was damped by κ. The parameter then kept a share α of its old value, added the update, and was clipped to the interval from −B to B. Proposition 1 shows that the parameter can never exceed the smaller of B and $\lambda/(1-\alpha)$. With the configured $\lambda = 0.001$ and $\alpha = 0.999$, $\lambda/(1-\alpha) = 1$, so $B = 0.05$ is the bound that binds. For the linear recurrence without clipping or damping, the mean bias settles at $(q - s)/2$, half of the prior's error.
Evidence status: `ANALYTIC` for the bound and the mean limit. The stability condition, $\rho(A) = 0.99876 < 1$, was checked numerically for this configuration only.
Do not accidentally conclude: The mean limit describes the average, not every noisy trajectory. The half-life of hundreds of updates is an asymptotic time scale, not a recovery bound. The buffers' context keys were empty, so this rule was not context-specific.
Try it: With $\lambda = 0.001$ and $\alpha = 0.999$, which number limits the parameter, B or $\lambda/(1-\alpha)$? (Worked in W02.)

P04. The margin: protection and its cost (Proposition 2)

Where we are: Proposition 2 (lines 113–168), The learning core (lines 345–361), and The cost of ranking protection (lines 524–529).
What the passage is doing: It proves when learning cannot reorder two options and tests both sides of that result.
In ordinary language: Suppose two options' prior scores come only from the input and the rules. If their gap is larger than 2B, no feedback sequence can reverse their order. That blocks an attacker who fabricates success reports. It also blocks true reports from fixing a prior that is wrong by more than 2B. Experiment B showed both effects. With the attacker's target 0.12 below the top (B6), no bounded learner ever ranked it first, while unbounded Thompson sampling selected it on 814 of 1,500 events on average. With a prior wrong by more than 2B (B4), no bounded arm reached the best option, a cost of +0.295 regret against Thompson sampling. Inside the margin, the target 0.03 below the top (B5) ranked first on 67.4% of second-half steps under the rule as designed. Experiment D's attacks gave the same pattern in D7 and D8.
Evidence status: `ANALYTIC` for Proposition 2; `MODELED_LOCKED` for Experiments B and D.
Do not accidentally conclude: The bound gives no rank guarantee inside the margin. It also gives none when learned transport costs feed the prior score, which needed the Max path with its flag set and would need a bound of its own.
Try it: Rules that are wrong by more than 2B cannot be corrected by bounded learning. What does the paper say such rules need instead?

P05. Verdict independence: tested, not proved

Where we are: Remarks 1 and 2, lines 113–168.
What the passage is doing: It separates what was proved from what was only tested.
In ordinary language: The safety verdict for an input was computed from the parsed input, the safety rules, and the anchor before any option was ranked. Learned parameters were not inputs to that computation, so feedback could not change a block or redirect verdict. The paper calls this a remark, not a proposition, because it rests on reading the code path, not on a proof. Check A3 tested it by forcing ranking parameters to each bound, and Check A9 by comparing verdicts with and without trained modules. In a separate post-hoc run, Check A3 caught M54, a mutant built to make a verdict read a learned value. Remark 2 reads the bound in control terms: learned parameters stay inside a fixed envelope, much as a Simplex monitor confines an unverified controller.
Evidence status: `MEASURED_DATED` for the checks; `INTERPRETIVE` for the control-theory reading.
Do not accidentally conclude: Passing checks on generated inputs does not prove independence for every input. None of these properties implies that a learned adjustment is correct.
Try it: Why is "no feedback sequence can change a verdict" stated as a remark rather than as Proposition 3?

P06. Tests that pass while learning does nothing

Where we are: Implementation invariants and what the checks missed, lines 302–344, and Fig 1.
What the passage is doing: It reports the study's clearest finding about testing.
In ordinary language: Before the study, the released system passed all 76 tests in its registry. One test checked that feedback engaged learning, but it asserted only that the stored value was above zero and inside its bound. After 1,500 consecutive successes, the rule as released had moved a ranking parameter to 0.87% of its bound, a value of 0.00044. A second retention stage in the released code shrank the update scale from 1 to about 0.001. The rule as designed reached 27% of the bound by event 100, 66% by event 200, and the bound at event 280. Reading the code found two more departures: damping was never applied, and substitution proposals ignored their routed direction. In B1, the rule as released had the same regret as the frozen ranking in every replicate. A test of update magnitude was then added.
Evidence status: `MEASURED_DATED`.
Do not accidentally conclude: Fixing the rule did not make momentum better than simpler learners; Experiment B found no advantage for it.
Try it: Besides "the value moved," what should a test of learning check? (Compare W03.)

P07. What the checks caught and what they missed

Where we are: Implementation invariants and what the checks missed (lines 302–344), and Invariants and the limits of their tests (lines 515–523).
What the passage is doing: It measures the test suite against deliberately inserted faults.
In ordinary language: A mutant is a copy of the code with one deliberate fault. The invariant checks caught 51 of 52 authored mutants. The one they missed, M01, removed a check that no input could expose. These mutants were not a held-out set: the agent that wrote the checks also wrote them, and the checks were revised twice after runs against them. Post hoc, 60 operator-level mutants were drawn at random. Of the 52 that changed behavior, the suite caught 38 (95% interval for the rate 60 to 83%), and 41 with the project's test registry. An exhaustive post-hoc pass over the safety and material-limit files gave 122 behavior-changing mutants. The suite caught 72, and 88 with the registry. Of the 34 missed, 28 were in material-limit thresholds and conditions that no test exercised. Earlier, Check A18 had found a transport solver that stopped at 300 iterations with mass out of balance by up to 1.11%.
Evidence status: `MEASURED_DATED`; the random draw and the exhaustive pass were post hoc.
Do not accidentally conclude: A catch rate of 51 of 52 does not mean high coverage. The suite tested the architecture's invariants, not the content of its safety rules or limits.
Try it: Why does having the same author write the checks and the mutants inflate a catch rate?

P08. Strong simple comparators beat the learned components

Where we are: The learning core (lines 345–361), Learned memories and the Hope block (lines 362–403), and Strong simple comparators (lines 530–541).
What the passage is doing: It compares each learned component with simple learners built to be strong.
In ordinary language: Multi-scale momentum showed no advantage over a bounded moving average. After a change in the best option (B3), the moving average switched in a median of 84 steps, against 462. Damping slowed adaptation and lowered regret in no scenario. The elastic meta-learner met its first two criteria, and the small residual memory helped when an earlier regime returned. A moving average keyed by the true label still had lower regret than both, usually by a large amount. The Hope block failed most of its pre-specified criteria. Its corrections shared an offset below the bound's lower limit, so every option's total clipped to −B and the ranking stayed at the prior. Post hoc, centering its output before the clip passed every pre-specified criterion against the current learner on 100 new seeds (Experiment H). It still had higher regret than the keyed average in D2, D3, D5, and D10.
Evidence status: `MODELED_LOCKED` for Experiments B, D, and E; Experiment H and the retrained blocks are post hoc.
Do not accidentally conclude: Nothing here concerns Hope on language tasks. The block had 21,859 parameters and ran on synthetic streams, and its centered read was not in the tested runtime. The momentum result concerns a scalar translation of the published optimizer.
Try it: Why does a shared negative offset erase the ranking once each correction is clipped on its own? (Worked in W04.)

P09. Whole systems: Experiments I, J, and K

Where we are: Integrated simulations (lines 239–273), Comparators (lines 274–285), and the three Experiment sections of Results (lines 425–464).
What the passage is doing: It tests the integrated decision-and-feedback loop against strong simple systems, then diagnoses and revises it.
In ordinary language: A simulated world offered three actions in three contexts. Each episode was one decision, scored by expected net utility: the modeled chance of success, or the stopping value, minus .02 per question. In Experiment I, the full pipeline scored 0.6080 against 0.6350 for a keyed moving average, a difference of −0.0270 over 32 fresh seed blocks. Records showed why: in 72.1% of episodes with several eligible actions, every correction equaled −0.05, so clipping erased the differences between actions. Experiment J centered the corrections and replaced hashed features. The revision beat the original by +0.0212 but still scored below a keyed discounted Beta mean (−0.0087). Experiment K found no established difference from a pooled moving average when one action was best everywhere (−0.0039, 95% CI −0.0109 to +0.0026). Separately coded quality-control programs, also written by coding agents, recomputed these results from the raw histories.
Evidence status: `MODELED_LOCKED` for the pre-specified contrasts. The diagnosis used records already used for testing, so it is not an estimate of a revised policy's benefit.
Do not accidentally conclude: The +0.0334 gain in J's shared family does not show a full-system advantage: that comparator was selected on the primary families. K's interval establishes neither equivalence nor pooled superiority. No person reimplemented the simulations or the analysis.
Try it: The null family gave every valid action the same success probability. Why could it register costs but no ranking benefit?

P10. The admission rule: few false alarms, low power, and a blind spot

Where we are: Component admission rule (lines 215–238), The admission rule (lines 404–424), Table 5, and Fig 2.
What the passage is doing: It measures how often the rule admits useless and useful components.
In ordinary language: A component had to show unique gain: removing it and refitting the combiner had to worsen the held-out Brier score, with a paired 95% interval above zero. Experiment C ran this step on simulated solvers whose information a was known. With five useless solvers, the rule admitted at least one in 1.2% of replicates at 331 groups per split. Power needed a sizable effect: 39.3% at a = 0.30 and 83.3% at a = 0.50. With two near-duplicate useful solvers, either one was admitted in only 2.0% of replicates. Removing one duplicate leaves the other, so almost nothing is lost. In the one earlier trial, on 1,288 human-rated solutions in 331 problem groups, no adapter was admitted.
Evidence status: `MODELED_LOCKED` for Experiment C; `MEASURED_DATED` for the earlier trial.
Do not accidentally conclude: A rejected component has not been shown to lack value. Given the power results, the earlier trial's rejection is weak evidence. Experiment C left out the universal and adapter-specific gates, so its rates are upper limits for the full rule.
Try it: The paper names one untested remedy for overlapping components. What is it? (Compare W06.)

Part 2: Detailed section readings (R01–R12)

Unit	Manuscript section	Lines	Where it is taught
R01	Introduction	1–58	P01
R02	Study objects and scope; Architecture	60–112	P02
R03	Update rule and guarantees	113–168	P03, P04, P05, W01–W03
R04	Component experiments	169–214	P08, S07
R05	Component admission rule	215–238	P10, W06
R06	Integrated simulations; Comparators; Internal pre-specification and intervals	239–300	P09, W05
R07	Implementation invariants and what the checks missed	302–344	P06, P07
R08	The learning core; Learned memories and the Hope block	345–403	P04, P08
R09	The admission rule	404–424	P10, W06
R10	Experiments I, J, and K	425–464	P09, W05
R11	Question selection, transport, and published methods	465–513	R11 notes below
R12	Discussion and Conclusions	514–634	R12 notes below, all lessons

The title page, abstract, Supporting information, Data and code availability, and Acknowledgments (lines 635–697) are covered by the exact-basis section, Part 3, and the global nonclaims.

R06 notes: how the experiments were pre-specified. For each experiment, criteria were written to a file before it ran, and the file hashes were recorded with the results. The hashes tied each file to its run but did not date it. An AI agent wrote each file in the session that ran the experiments, and no person reviewed the files before the runs. The paper therefore calls this internal pre-specification. Pointwise 95% intervals do not give simultaneous coverage across contrasts.

R11 notes: question selection, transport, and published methods.

- **Expected free energy (Experiment F):** it reduced final decision loss relative to fixed-order questioning (−0.0039). It showed no established benefit over bounded contrasts, the current selector (−0.0003, 95% CI −0.0027 to +0.0021).
- **Transport:** it beat a greedy baseline on some measures but had lower point estimates than the exact allocator on top-option agreement. The admission rule asked transport to outperform the exact allocator, and it did not. Learning cost weights lowered held-out allocation loss from 0.660 to 0.248, but that test let all five weights learn. The system let only the two non-safety weights learn, and a post-hoc rerun with only those two changed loss about a tenth as much.

- **H-JEPA (Experiments G1 and G2):** its official planner succeeded on 48 of 50 tasks. It takes pixel observations and returns continuous actions, so it did not fit Level 3 (Planner). A frozen forbidden-zone veto cut steps inside the zone from 379 to zero, and success fell from 96.0% to 76.0%. A veto blocks unsafe motion but does not supply a plan.
- **RLCP (Experiment G3):** applied to a fine-tuned Laya checkpoint, it failed three of its four pre-specified measures. A fresh linear probe read the city at 99.3%, against 74.0% for the original checkpoint. The RLCP checkpoint followed a context naming the wrong country less often than the original, 18.1% against 84.7%, the opposite of the architecture's aim.
- **Laya on PIQA:** a fine-tuned head beat a lexical baseline by +0.080 in accuracy. Overlap with its pretraining data cannot be excluded, so the paper does not count that gain as evidence of generalization.

R12 notes: discussion and conclusions. The escape room was a small illustrative simulator with four hand-written scripts, and the paper draws no inference from it. The anchored script escaped in every run, and the naive script ended in a fatal violation in 90.7% of runs. Those rates follow from how the scripts and rules were written. The paper relates its results to fault-tolerant control by analogy: the frozen layer acts like a supervisory monitor, and the bounded layer adapts inside a fixed envelope. Its design priorities are to test update magnitude and rule content and to preserve useful contrasts within the bound. They also include comparing learned memories with strong keyed and pooled learners and evaluating admission rules that account for overlapping components. Whether frozen supervisory rules add measurable safety on real workloads remains to be tested.

Part 3: Supplement modules (S01–S07)

- **S01: S1 Appendix (Implementation checks, authored mutants, and mean-recurrence stability).** It lists the implementation checks with sample sizes and failure definitions, and the 52 authored mutants with the checks that caught them. It also derives the mean recurrence behind Proposition 1.
- **S02: S2 Appendix (Experiment I design and details).** It gives the pipeline and world design, development and confirmation, all seven confirmation configurations, and learning curves. It also reports the invalid allocations: 201 of 30,720 episodes in the full arm, which concerned approximate resource conservation, not observed physical harm.
- **S03: S3 Appendix (Experiment J design and details).** It gives the diagnosis of Experiment I, the revised read and token memory, and the audit and allocation changes. It also covers the two new families, candidates and selection, and results by family, including stress and asynchronous.
- **S04: S4 Appendix (Experiment K design and details).** It gives the design, precision plan, selection, and confirmation of the pooled-comparator study. Its pre-specified primary contrast was full minus pooled simple in the shared family.
- **S05: S5 Appendix (Simulation reporting and computational environment).** It gives a simulation-study reporting summary, hardware and software, and a Linux portability check. It also reports latency and run times and the post-hoc sweep of the simple learners' rates.
- **S06: S6 Appendix (Illustrative escape-room simulator).** The facilities, veto rules, four scripted policies, and their outcome rates.

- **S07: S7 Appendix (Designs of Experiments A to H and deviations).** Specifications of the optional components; the aims, data-generating mechanisms, estimands, methods, and performance measures of Experiments A to H; interval conventions; and every deviation, including the post-hoc analyses.

  S1 File, the replication archive, holds the tested runtime, the scripts, and the result records. Its minimal data set lets a supplied script recompute the main Experiment I, J, and K results. The raw histories are deposited at Zenodo (https://doi.org/10.5281/zenodo.24117501).

Part 4: Worked examples (W01–W06)

W01: Checking the margin

Inputs from the paper: for ranking, B = 0.05, so 2B = 0.10. Scenario B5 placed the attacker's target 0.03 below the top option, and B6 placed it 0.12 below.

1. Compare each gap with 2B. In B5, 0.03 is less than 0.10. In B6, 0.12 is more than 0.10.
2. Apply Proposition 2 to B6. Learning can lower the top option by at most 0.05 and raise the target by at most 0.05. The top option stays ahead by at least 0.12 − 0.10 = 0.02.
3. Apply it to B5. The gap is inside the margin, so the proposition says nothing.
4. Compare with the simulations. In B6, no bounded arm ever ranked the target first. In B5, it ranked first on 67.4% of second-half steps under the rule as designed.

W02: Which bound binds (Proposition 1)

Inputs from the paper: $\lambda = 0.001$ and $\alpha = 0.999$.

1. Compute $1 - \alpha = 0.001$.
2. Compute $\lambda/(1 - \alpha) = 0.001/0.001 = 1$.
3. The parameter is limited by the smaller of B = 0.05 and 1, so B binds.
4. For the mean limit, $\lambda(q - s)/(1 - \alpha + \lambda) = 0.001(q - s)/0.002 = (q - s)/2$. In the linear recurrence, learning removes half of the prior's error in mean.

W03: Why the rule as released barely moved

Inputs from the paper: the released code added a second retention stage that replaced θ with $\alpha 2\theta + \eta 2\delta/\alpha$, where $\alpha 2 = 0.97$ and $\eta 2 = 0.03$. Its fixed point is δ/α.

1. Each update δ is at most $\lambda = 0.001$ in size, and $\alpha = 0.999$, so the fixed point δ/α is at most about 0.001.
2. The scale in Proposition 1 therefore fell from $\lambda/(1 - \alpha) = 1$ to about $\lambda/\alpha \approx 0.001$.
3. After 1,500 consecutive successes, the paper reports 0.87% of the bound. Since 0.87% of 0.05 is 0.000435, this matches the reported value of 0.00044.
4. A test that only asks whether the value is above zero passes. The project then added a test of update magnitude, which checks how far the value moves.

W04: How a shared offset erases a ranking, and how centering keeps it

Illustrative inputs (not from the paper): three actions with raw corrections −0.09, −0.08, and −0.07, and B = 0.05.

1. Clip each correction on its own to the interval from −0.05 to 0.05. All three become −0.05.
2. The corrections are now equal, so they no longer separate the actions. The ranking stays at the prior. Experiment I's records showed this in 72.1% of the episodes with several eligible actions.
3. Center instead: subtract the mean, −0.08. The corrections become −0.01, 0.00, and +0.01.
4. These fit inside the bound, so the differences survive. Experiment J's read summed each action's contributions, subtracted their mean, and contracted the vector only if needed to fit the bound.

W05: Reading an interval against the superiority criterion

Inputs from the paper: practical superiority required a lower 95% limit above .01, zero forbidden attempts, and zero invalid allocations.

1. Experiment I: −0.0270 (−0.0300 to −0.0241). The whole interval is below zero, so the simpler system scored higher. The full arm also made 201 invalid allocations.
2. Experiment J against the original: +0.0212 (+0.0172 to +0.0251). The lower limit is above .01, with zero forbidden attempts and zero invalid allocations, so this criterion was met.
3. Experiment J against the simple comparator: −0.0087 (−0.0136 to −0.0039). This second criterion failed.
4. Experiment K: −0.0039 (−0.0109 to +0.0026). The interval includes zero, so it establishes neither equivalence nor pooled superiority.

W06: Why near duplicates defeat a unique-gain rule

Inputs from the paper: at a = 0.50 and 331 groups per split, one useful solver alone was admitted in 83.3% of replicates. With two near duplicates, at least one was admitted in 2.0%.

1. The unique-gain step removes one solver, refits the combiner, and asks whether the held-out Brier score worsens.
2. Remove one duplicate. The other still carries nearly the same information, so the score barely changes.
3. The same holds for the other duplicate, so neither shows unique gain and both are usually rejected.
4. Shared noise did the same to five useful solvers: admission of at least one fell from 78.7% to 7.0%. The paper suggests, but did not test, removing correlated components as a group.

Part 5: Self-test (T01–T12)

Questions T01 to T04 are core questions on the main results. T05 to T08 are boundary questions on what the paper does not show, and T09 to T12 apply the ideas to new cases. Numbers in the transfer questions are illustrative.

T01. Core: What does Proposition 2 guarantee?

- A. Learning always moves options toward their true values.
- B. If two prior scores differ by more than 2B, no feedback sequence can reverse their order.
- C. Fabricated reports can never move any option.
- D. The rules' prior scores are correct.

Answer: B. The guarantee covers gaps larger than 2B, for priors that read no learned value. Inside the margin it says nothing, and it never says the priors are right.

T02. Core: What was the primary estimand of Experiments I to K?

- A. The rate of forbidden actions.
- B. The difference in expected net utility per episode between the full system and a simpler system held to the same bound.
- C. The physical success rate of repairs.
- D. The number of tests passed.

Answer: B. Expected net utility was a modeled score, not a physical success rate.

T03. Core: What did Experiment I find?

- A. The full pipeline scored higher than the simple comparator.
- B. The two scored the same.
- C. The full pipeline scored lower: −0.0270, 95% CI −0.0300 to −0.0241.
- D. The experiment was stopped early.

Answer: C. The selected keyed moving average scored 0.6350 against 0.6080 for the full pipeline.

T04. Core: Why did the update rule as released change no decision?

- A. A second retention stage shrank the update scale to about 0.001.
- B. The bound B was set to zero.
- C. The tests blocked every update.
- D. The anchor's hash changed.

Answer: A. The parameter reached only 0.87% of its bound after 1,500 consecutive successes.

T05. Boundary: The anchor's hash is pinned. What does that show?

- A. The safety rules are correct.
- B. The rule file has not changed.
- C. The system is safe in practice.
- D. The rules were reviewed by experts.

Answer: B. A pin protects a file without validating its content, and the hard-block list has had no expert review.

T06. Boundary: Experiment K's interval runs from −0.0109 to +0.0026. Which conclusion does it support?

- A. The pooled learner is better.
- B. The full pipeline is better.
- C. The two are equivalent.
- D. None of these; the interval establishes no difference and no equivalence.

Answer: D. An interval that includes zero shows no established difference, and it does not establish equivalence either.

T07. Boundary: The centered Hope block passed every pre-specified criterion against the current learner in Experiment H. What follows?

- A. Hope blocks work on language tasks.
- B. Centering helped this compact block on these synthetic streams, in a post-hoc change.
- C. The block beat the keyed moving average everywhere.
- D. The block was part of the tested runtime.

Answer: B. The change was post hoc, the block still trailed the keyed average in D2, D3, D5, and D10, and its centered read was not in the tested runtime.

T08. Boundary: In the earlier admission trial, no adapter was admitted. What does that show?

- A. The adapters had no value.
- B. Little, because the rule needed large samples to admit a component with a small real benefit.
- C. The admission rule is broken.
- D. The data were wrong.

Answer: B. The paper calls this rejection weak evidence that those adapters lack value.

T09. Transfer: A new system uses B = 0.02, and two options' prior scores differ by 0.05. Can fabricated reports reorder them?

- A. Yes, always.
- B. No, because the gap is larger than 2B = 0.04.
- C. Only if the priors read learned values.
- D. It depends on the learning rate.

Answer: B. The gap exceeds the margin, so by Proposition 2 the order holds, provided the priors read no learned value.

T10. Transfer: A team's only learning test checks that a weight is nonzero after training. What does the paper suggest adding?

- A. More tests of the same kind.
- B. A test that the update is large enough to change a decision.
- C. A larger bound.
- D. Nothing; the test is enough.

Answer: B. All 76 tests passed while learning changed no decision. Tests should measure the intended effect as well as the mechanics.

T11. Transfer: You compare a complex recommender with a moving average, but you let the moving average ignore the bound. What is wrong?

- A. Nothing.
- B. The comparison mixes the learner with the protection; both arms should face the same bound and the same checks.
- C. Moving averages cannot be bounded.
- D. The complex system should also ignore the bound.

Answer: B. In the paper, each selected comparator used the same correction bound and frozen checks, so the comparison isolated whole-system differences.

T12. Transfer: Two sensors carry nearly the same signal, and a leave-one-out test rejects both. What would the paper suggest trying?

- A. Admitting both without a test.
- B. Removing correlated components as a group, which the paper names but did not test.
- C. Lowering the bound.
- D. Dropping the test.

Answer: B. A low leave-one-out gain does not distinguish a useless component from a redundant one.

Teach it back

In one sentence, say what the bound does and what the experiments found. A good answer: a bound on learning locks in rule-based gaps above 2B for true and fabricated reports alike, and complex components did not beat equally bounded simple learners.

Global nonclaims, in the paper's words

- "All evidence was computational; no physical system was tested."
- "We make no claim about real-world outcomes, and we report negative results as results"
- "No component received deployment admission, which required real-outcome evidence that this study did not have"
- "Shared frozen checks give identical safety outcomes by construction, and zero observed violations do not establish zero risk."
- "The system was built and tested by one author with AI agents, and no one has replicated it."